To implement this correctly for a professional FPGA project, you should separate your **Design RTL** (which is synthesized into hardware) from your **Testbench** (which only runs in simulation).

1. The Design RTL (top_module.v)

This is the file you would add to your Vivado or ISE project. It contains the BSCANE2 primitive which acts as the bridge between the JTAG cable and your internal logic. Note the use of the SEL signal to ensure your logic only reacts when the specific JTAG User Chain is active.

Verilog

```
`timescale 1ns / 1ps

module my_jtag_logic (
    input wire clk,
    input wire rst_n
);

    // BSCANE2 Signals
    wire capture, drck, reset, runtest, sel, shift, tck, tdi, update;
    reg  tdo_user;

    // Example Internal Logic: An 8-bit Shift Register controlled by JTAG
    reg [7:0] my_data_reg;

    // Instantiate BSCANE2 (The Hardware Bridge)
    // JTAG_CHAIN 1 corresponds to the USER1 Instruction
    BSCANE2 #(
        .JTAG_CHAIN(1)
    ) bscan_inst (
        .CAPTURE(capture),
        .DRCK(drck),
        .RESET(reset),
        .RUNTEST(runtest),
        .SEL(sel),
        .SHIFT(shift),
        .TCK(tck),
        .TDI(tdi),
        .TMS(),       // TMS is handled internally by the TAP
        .UPDATE(update),
        .TDO(tdo_user)
```

```
  );

  // logic to interact with JTAG
  always @(posedge tck) begin
    if (sel && shift) begin
      // Shift data in from JTAG TDI
      my_data_reg <= {tdi, my_data_reg[7:1]};
    end
  end

  // Feed the LSB of our register back to JTAG TDO
  always @(*) begin
    tdo_user = my_data_reg[0];
  end

endmodule
```

2. The Testbench (tb_top.v)

This file is **never synthesized**. It contains the JTAG_SIME2 model you uploaded, which mimics the physical JTAG pins. We use a compiler directive (SIMULATION) to ensure the simulation model is only "seen" by the simulator.

Verilog

```
`timescale 1ns / 1ps

// Define SIMULATION if your tool doesn't do it automatically
`define SIMULATION

module tb_top();

  // Physical JTAG signals (Wires that don't exist in the FPGA fabric)
  reg  tck_pin;
  reg  tms_pin;
  reg  tdi_pin;
  wire tdo_pin;

  // 1. Instantiate the Simulation Model
  // This is the "JTAG Cable" side.
  `ifdef SIMULATION
```

```
    JTAG_SIME2 #(
        .PART_NAME("7K325T")
    ) jtag_cable_sim (
        .TCK(tck_pin),
        .TMS(tms_pin),
        .TDI(tdi_pin),
        .TDO(tdo_pin)
    );
`endif

// 2. Instantiate the Design Under Test (DUT)
my_jtag_logic dut (
    .clk(1'b0), // Not used in this basic JTAG example
    .rst_n(1'b1)
);

// --- Clock Generation ---
initial tck_pin = 0;
always #50 tck_pin = ~tck_pin;

// --- JTAG Tasks ---
task jtag_step(input tms, input tdi);
    begin
        tms_pin = tms;
        tdi_pin = tdi;
        @(posedge tck_pin);
        #1;
    end
endtask

task jtag_reset();
    begin
        repeat(5) jtag_step(1, 0);
        jtag_step(0, 0); // Idle
    end
endtask

task jtag_update_ir(input [5:0] instr);
    integer i;
    begin
        jtag_step(1, 0); // Sel-DR
        jtag_step(1, 0); // Sel-IR
        jtag_step(0, 0); // Cap-IR
```

```
        jtag_step(0, 0); // Shift-IR
        for (i=0; i<5; i=i+1) jtag_step(0, instr[i]);
        jtag_step(1, instr[5]); // Exit1-IR
        jtag_step(1, 0); // Update-IR
        jtag_step(0, 0); // Idle
    end
  endtask

  // --- Stimulus ---
  initial begin
    tms_pin = 1; tdi_pin = 0;
    #200;

    jtag_reset();

    // Load USER1 Instruction (0x02) to activate BSCANE2
    jtag_update_ir(6'h02);

    // Shift 8 bits into our 'my_data_reg' via JTAG
    jtag_step(1, 0); // Sel-DR
    jtag_step(0, 0); // Cap-DR
    jtag_step(0, 0); // Shift-DR

    repeat(7) jtag_step(0, 1); // Shift in 1s
    jtag_step(1, 1);           // Last bit + Exit-DR

    jtag_step(1, 0); // Update-DR
    jtag_step(0, 0); // Idle

    #500;
    $display("Simulation complete.");
    $finish;
  end

endmodule
```

Summary of the Strategy:

1. **Macro Protection:** The `ifdef SIMULATION block ensures that the synthesizer (which is building hardware) ignores the JTAG_SIME2 component because that component contains code that cannot be turned into physical gates.
2. **Internal vs External:** JTAG_SIME2 represents the pins on the outside of the chip. BSCANE2 represents the connection on the inside of the chip.

3. **Synthesis:** When you run synthesis in Vivado, only my_jtag_logic is processed. The JTAG_SIME2 remains a "virtual" component used only to verify your logic during development.